

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 9 · Ficheros binarios, serialización y NIO

Flujos de bytes: FileInputStream y FileOutputStream

Flujos de datos: DataInputStream y DataOutputStream

Serialización: ObjectOutputStream, ObjectInputStream, Serializable y transient

Lectura/escritura de texto con buffer: BufferedReader y BufferedWriter

java.nio.file: Path, Files (readAllLines, write, walk)

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Ficheros binarios y serialización

Parte 1: Opción múltiple y Verdadero/Falso

4 puntos — 0,5 cada pregunta

1. ¿Qué interfaz debe implementar una clase para poder guardar sus objetos con `ObjectOutputStream`?
- a) `Cloneable`
 - b) `Comparable`
 - c) `Serializable`
 - d) `Iterator`

RESPUESTA / SOLUCIÓN

c) `Serializable` — Es una interfaz de marcado (sin métodos) que indica que el objeto puede serializarse.

-
2. Un campo declarado como `transient` no se guarda al serializar el objeto.

RESPUESTA / SOLUCIÓN

Verdadero — `transient` excluye el campo de la serialización; al deserializar queda con el valor por defecto (0, null, false...).

-
3. Si guardamos un `int` con `out.writeInt(7)`, ¿qué método hay que usar para leerlo en el mismo orden?
- a) `readDouble()`
 - b) `readInt()`
 - c) `readLine()`
 - d) `readObject()`

RESPUESTA / SOLUCIÓN

b) `readInt()` — Los flujos de datos (`DataInputStream/DataOutputStream`) exigen leer los tipos en el mismo orden en que se escribieron.

-
4. Con `try-with-resources` (`try (BufferedReader br = ...) { ... }`), al terminar el bloque, ¿qué ocurre con `br`?
- a) hay que cerrar `br` manualmente
 - b) `br` se cierra automáticamente
 - c) `br` se borra del disco
 - d) se lanza `IOException` siempre

RESPUESTA / SOLUCIÓN

b) `br` se cierra automáticamente — `try-with-resources` cierra todos los recursos declarados entre paréntesis al finalizar el bloque, incluso si hay una excepción.

-
5. `Files.write(Path, List<String>)` permite escribir un fichero de texto a partir de una lista de líneas.

RESPUESTA / SOLUCIÓN

Verdadero — `java.nio.file.Files.write()` crea (o sobrescribe) el fichero y escribe cada `String` de la lista como una línea.

6. ¿Qué excepción lanza `ObjectOutputStream` al intentar serializar un objeto que NO implementa `Serializable`?

- a) `IOException`
- b) `NotSerializableException`
- c) `ClassNotFoundException`
- d) `FileNotFoundException`

RESPUESTA / SOLUCIÓN

b) `NotSerializableException` — Es una subclase de `IOException` que se lanza cuando un objeto no es serializable.

7. El método `read()` de `FileInputStream` devuelve un valor. ¿Cuál?

- a) un `char`
- b) un `int` con el byte leído (o -1 si es fin de fichero)
- c) un `boolean`
- d) un `String` con la línea

RESPUESTA / SOLUCIÓN

b) un int con el byte leído (o -1 si es fin de fichero) — Por eso el bucle típico de copia es while ((b = in.read()) != -1).

-
8. Un fichero binario puede leerse con Scanner igual que un fichero de texto.

RESPUESTA / SOLUCIÓN

Falso — Scanner está pensado para texto; para datos binarios hay que usar flujos de bytes o de datos (FileInputStream, DataInputStream, ObjectInputStream...).

Parte 2: Análisis y depuración

3 puntos

1. El siguiente programa tiene 2 errores. Encuéntralos, explica por qué fallan y propón la corrección. (1,5 ptos)

```
public class Guardar {  
    public static void main(String[] args) throws IOException {  
        Persona p = new Persona("Ana", 22);  
        ObjectOutputStream out =  
            new ObjectOutputStream(new FileOutputStream("p.dat"));  
        out.writeObject(p);  
    }  
}
```

RESPUESTA / SOLUCIÓN

1. La clase Persona no implementa Serializable: al ejecutar `out.writeObject(p)` se lanza `NotSerializableException`. Corrección: `public class Persona implements Serializable { ... }` (y declarar `serialVersionUID`).

2. El flujo `out` nunca se cierra: los datos pueden no volcarse al disco y el recurso queda abierto. Corrección: usar `try-with-resources`:

```
try (ObjectOutputStream out =  
    new ObjectOutputStream(new FileOutputStream("p.dat"))) {  
    out.writeObject(p);  
}
```

-
2. Analiza la siguiente escritura y determina qué instrucciones de lectura recuperan los datos y cuántos bytes ocupa cada valor. (1,5 ptos)

```
DataOutputStream out =  
    new DataOutputStream(new FileOutputStream("datos.dat"));  
out.writeInt(7);  
out.writeDouble(2.5);  
out.writeUTF("Hola");  
out.close();
```

RESPUESTA / SOLUCIÓN

Los datos se leen en el MISMO orden en que se escribieron:

```
int i = in.readInt();    // 7 (4 bytes)
```

```
double d = in.readDouble(); // 2.5 (8 bytes)
```

```
String s = in.readUTF();  // "Hola" (2 bytes de longitud + 4 de texto = 6 bytes)
```

Tamaño total del fichero: $4 + 8 + 6 = 18$ bytes.

Parte 3: Ejercicios de programación

8 puntos

1. Crea la clase **Persona** (nombre, edad) que implemente **Serializable** y un programa que guarde una lista de 3 personas en **personas.dat** con **ObjectOutputStream** y después la lea con **ObjectInputStream** mostrándola por pantalla. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.io.*;
import java.util.*;

public class GuardarPersonas {

    static class Persona implements Serializable {
        private static final long serialVersionUID = 1L;
        String nombre; int edad;

        Persona(String nombre, int edad) { this.nombre = nombre; this.edad = edad; }

        public String toString() { return nombre + ", " + edad + " años"; }
    }

    public static void main(String[] args) {
        List<Persona> personas = List.of(
            new Persona("Ana", 22),
            new Persona("Luis", 19),
            new Persona("Eva", 24));

        try (ObjectOutputStream out =
            new ObjectOutputStream(new FileOutputStream("personas.dat"))) {
            out.writeObject(personas);
        } catch (IOException e) {
            System.out.println("Error al guardar: " + e.getMessage());
        }

        try (ObjectInputStream in =
            new ObjectInputStream(new FileInputStream("personas.dat"))) {
            List<Persona> leidas = (List<Persona>) in.readObject();

            for (Persona p : leidas) System.out.println(p);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error al leer: " + e.getMessage());
        }
    }
}
```


}

2. Escribe un programa que copie un fichero binario (por ejemplo foto.jpg) a otro

(copia_foto.jpg) leyendo byte a byte con FileInputStream y escribiendo con FileOutputStream. Muestra cuántos bytes se copiaron. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.io.*;

public class CopiarFichero {

    public static void main(String[] args) {

        File origen = new File("foto.jpg");
        File destino = new File("copia_foto.jpg");
        int contador = 0;

        try (FileInputStream in = new FileInputStream(origen);
            FileOutputStream out = new FileOutputStream(destino)) {

            int b;

            while ((b = in.read()) != -1) {

                out.write(b);

                contador++;

            }

        } catch (IOException e) {

            System.out.println("Error: " + e.getMessage());

        }

        System.out.println("Copiados " + contador + " bytes");

    }

}
```

-
3. Escribe un programa que pida 5 números double por teclado, los guarde en numeros.dat con DataOutputStream y después los lea con DataInputStream mostrando la suma y la media. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.io.*;
import java.util.Scanner;

public class DatosBinarios {

    public static void main(String[] args) throws IOException {

        Scanner sc = new Scanner(System.in);

        try (DataOutputStream out =
            new DataOutputStream(new FileOutputStream("numeros.dat"))) {

            for (int i = 0; i < 5; i++) {

                System.out.print("Numero " + (i + 1) + ": ");

                out.writeDouble(sc.nextDouble());

            }

        }

        double suma = 0; int contador = 0;

        try (DataInputStream in =
            new DataInputStream(new FileInputStream("numeros.dat"))) {

            while (in.available() > 0) {

                suma += in.readDouble();

                contador++;

            }

        }

        System.out.println("Suma: " + suma);

        System.out.println("Media: " + (suma / contador));

    }

}
```

4. } Con **java.nio.file**: lee el fichero **alumnos.csv** (formato **nombre;nota**, una línea por alumno) con **Files.readAllLines**, calcula la nota media y escribe el resultado en **media.txt** con **Files.write**. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.nio.file.*;
import java.io.IOException;
import java.util.*;

public class NotaMedia {

    public static void main(String[] args) throws IOException {

        Path ruta = Paths.get("alumnos.csv");

        List<String> lineas = Files.readAllLines(ruta);

        double suma = 0; int n = 0;

        for (String linea : lineas) {

            String[] campos = linea.split(",");

            if (campos.length == 2) {

                suma += Double.parseDouble(campos[1].trim());

                n++;

            }

        }

        double media = (n > 0) ? suma / n : 0;

        Files.write(Paths.get("media.txt"),

            Arrays.asList("Alumnos: " + n, "Nota media: " + media));

        System.out.println("Media calculada: " + media);

    }

}
```

Parte 4: Pregunta teórica

1 punto

1. Explica las diferencias entre ficheros de texto y ficheros binarios, y entre serializar objetos y escribir los datos manualmente. ¿Qué ventajas aporta `java.nio.file` frente a `java.io.File`?

RESPUESTA / SOLUCIÓN

Texto: legible por humanos, se lee con `Scanner/BufferedReader`; binario: compacto y sin conversión de tipos, se lee con flujos de bytes o de datos.

Serializar guarda el objeto completo (estado y estructura) con un solo `writeObject/readObject`, pero el formato depende de la clase y su versión (por eso existe `serialVersionUID`); escribir manualmente da control total del formato y es más estable entre versiones, pero hay que escribir cada campo.

`java.nio.file` (`Path`, `Files`) ofrece métodos de alto nivel (`readAllLines`, `write`, `walk`, `copy`, `move...`), trata las rutas de forma multiplataforma y permite operaciones atómicas; `java.io.File` es más antiguo, con menos funcionalidad y métodos que devuelven boolean sin informar del error.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)